

Computação Gráfica

TRANSFORMAÇÕES GEOMÉTRICAS 2D

Translação: $x' = x + tx$

$$y' = y + ty$$

- Mantém ângulos
- Mantém comprimentos
- Mantém paralelismos

$$T(tx, ty) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ tx & ty & 1 \end{bmatrix}$$

$$P' = P + [tx \ ty]$$

$$[T(tx, ty)]^{-1} = T(-tx, -ty)$$

Ponto arbitrário: $T(-tx, -ty) \cdot S(sx, sy) \cdot T(tx, ty)$

Coordenadas homogêneas

$$P' = P \cdot T(tx, ty) \Rightarrow [x \ y \ 1] \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ tx & ty & 1 \end{bmatrix}$$

Rotação: $x' = x \cdot \cos \alpha + y \cdot \sin \alpha$

$$y' = x \cdot \sin \alpha + y \cdot \cos \alpha$$

- Mantém ângulos
- Mantém comprimentos
- Mantém paralelismos

$$R(\alpha) = \begin{bmatrix} \cos \alpha & \sin \alpha & 0 \\ -\sin \alpha & \cos \alpha & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$P' = P \cdot \begin{bmatrix} \cos \alpha & \sin \alpha \\ -\sin \alpha & \cos \alpha \end{bmatrix}$$

$$[R(\alpha)]^{-1} = R(-\alpha)$$

NOTA: $\sin(-\alpha) = -\sin \alpha$
 $\cos(-\alpha) = \cos \alpha$

Coordenadas homogêneas

$$P' = P \cdot R(\alpha) \Rightarrow [x \ y \ 1] \begin{bmatrix} \cos \alpha & \sin \alpha & 0 \\ -\sin \alpha & \cos \alpha & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Ponto arbitrário: $T(-tx, -ty) \cdot R(\alpha) \cdot T(tx, ty)$

Mudança Escala: $x' = x \cdot sx$

$$y' = y \cdot sy$$

- Não mantém ângulos
- Não mantém comprimentos
- Mantém paralelismos

$$S(sx, sy) = \begin{bmatrix} sx & 0 & 0 \\ 0 & sy & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$P' = P \cdot \begin{bmatrix} sx & 0 \\ 0 & sy \end{bmatrix}$$

$$[S(sx, sy)]^{-1} = S(1/sx, 1/sy)$$

P. arbitrário: $T(-tx, -ty) \cdot S(sx, sy) \cdot T(tx, ty)$

Coordenadas homogêneas

$$P' = P \cdot S(sx, sy) \Rightarrow [x \ y \ 1] \begin{bmatrix} sx & 0 & 0 \\ 0 & sy & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Shearing: $x' = x + y \cdot ky$

$$y' = x \cdot ky + y$$

$$Sh(kx, ky) = \begin{bmatrix} 1 & ky & 0 \\ kx & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Comutativa qd: concatenamos funções do mesmo tipo

Composição ou concatenação transformações geométricas

Translação e Rotação são **aditivas**

$$\text{Ex.: } T(tx, ty) \cdot T(vx, vy) = T(tx + vx, ty + vy)$$

$$R(\alpha) \cdot R(\beta) = R(\alpha + \beta)$$

concatenamos uma escala e uma mudança de escala =

Mudança Escala é **multiplicativa**

$$\text{Ex.: } S(sx, sy) \cdot S(kx, ky) = S(sx \cdot kx, sy \cdot ky)$$

Clipping window

Área retangular cl todos pontos aos eixos principais que contém porção da cena que vai ser representada na imagem

Mudança de Escala

Viewport

Área retangular cl todos pontos aos eixos principais que contém a imagem correspondente à porção da cena que está contida na clipping window

Sem deformação

$$sx = sy$$

Razão aspecto = h/a

$$T(-x_{wmin}, -y_{wmin}) \cdot S(sx, sy) \cdot T(x_{vmin}, y_{vmax})$$

$$sx = \frac{x_{vmax} - x_{vmin}}{x_{wmax} - x_{wmin}}$$

$$sy = \frac{y_{vmax} - y_{vmin}}{y_{wmax} - y_{wmin}}$$

Ex

1

TRANSFORMAÇÕES GEOMÉTRICAS 3D

Translação: $[n' \ y' \ z' \ 1] = [n \ y \ z \ 1] \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ t_x & t_y & t_z & 1 \end{bmatrix}$

Mudança Escala: $[n' \ y' \ z' \ 1] = [n \ y \ z \ 1] \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$

Rotação: Positivas em torno eixo quando efetuado no sentido contrário aos ponteiros relógio.

- rotar Y 90° em torno X faz Y coincidir c/ Z
- rotar Z 90° em torno Y faz Z coincidir c/ X
- rotar X 90° em torno Z faz X coincidir c/ Y

$$R_x(\alpha) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos(\alpha) & \sin(\alpha) & 0 \\ 0 & -\sin(\alpha) & \cos(\alpha) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$R_y(\alpha) = \begin{bmatrix} \cos(\alpha) & 0 & -\sin(\alpha) & 0 \\ 0 & 1 & 0 & 0 \\ \sin(\alpha) & 0 & \cos(\alpha) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$R_z(\alpha) = \begin{bmatrix} \cos(\alpha) & \sin(\alpha) & 0 & 0 \\ -\sin(\alpha) & \cos(\alpha) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Rotação em torno eixo arbitrário:

- fazer mudança p/ referencial

definido por

- efetuar rotação
- desfazer mudança inicial

- uma direção e um ponto qualquer no espaço
- 2 pontos no espaço
- outra forma equivalente

Exemplos: $T(-n_0, -y_0, -z_0) \cdot R_z(B) \cdot R_x(\alpha) \cdot R_z(\alpha) \cdot R_x(-\alpha) \cdot R_z(-B) \cdot T(n_0, y_0, z_0)$

↑
eixo passa origem
eixo fixo coincidente c/ eixo (z)

↑
eixo rotacionado sobre 1/2
coincidente com z

C/ esta transformação composta, inversa da 1ª, o eixo volta à posição original

PROJEÇÕES PLANARES

→ A projeção planar geométrica de um ponto é a interseção da reta projetante desse ponto c/ o plano projeção

→ posição deste plano em relação aos eixos do referencial 3D

→ se paralelos entre si
→ existe direção projeção

→ não paralelos entre si
→ existe ponto convergência

Projeção Paralela - Visão paralelopipédica

- Plano Projeção não pode ser paralelo à direção projeção
- Mantém paralelismos

$DP \perp PP$ ($DP \perp PP$)

Projeção Perspectiva - Visão pirâmide

- Projeção retas paralelas a uma direção D não paralela ao plano projeção, convergem p/ ponto fuga
- Se D for direção principal, DF é PF principal
Caso contrário $\Rightarrow$ PF secundário
- Classificadas em função n° de PFs principais

$CP \perp PP$ c/ $CP \perp PP$

1FP: PP paralelo a um plano principal
interseção: 1 eixo principal

2FP: PP paralelo a um plano principal
interseção: 2 eixos principais

3FP: PP paralelo interseção: 3 eixos principais

Ortográficas / Ortogonais
($DP \perp PP$)

PP // PP paralelo
Vistas (altos/plantas)
Axonométricas
PP não // Plano Principal

- Isométrica - DP faz ângulos $\approx 30^\circ$ c/ 3 eixos principais
- Dimétrica - DP faz ângulos $\approx 30^\circ$ c/ 2 eixos principais
- Trimétrica - DP faz ângulos $\approx 30^\circ$ c/ 3 eixos principais

Obliques
(DP não $\perp PP$)

- Cavaleira $\angle$ entre DP e PP = 45°
- Gabinete $\angle$ entre DP e PP = arctan 0.5
- Geral

Profundidade de correspondência à direção perpendicular ao plano projeção

Obtidas por projeção obliques do plano projeção

EM MATRIZ:

PP Ortográficas:

plano XY:
$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

plano YZ:
$$\begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

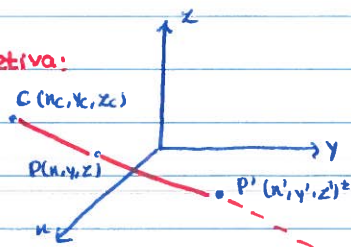
plano XZ:
$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

PP Obliques: Eq. Paramétricas: $n' = n + n_p \cdot u$
vetor $y' = y + y_p \cdot u$
(x_p, y_p, z_p) = direção projeção $z' = z + z_p \cdot u$

u = 0: interseção perpendicular c/ PP = P. Proj.

$$[n' \ y' \ z' \ 1] = [n \ y \ z \ 1] \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ -y_p/z_p & -y_p/z_p & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

P Perspectiva:



$$P = \begin{bmatrix} -x_c & 0 & 0 & 0 \\ 0 & -x_c & 0 & 0 \\ n_c & y_c & 0 & 1 \\ 0 & 0 & 0 & -x_c \end{bmatrix}$$

A COR

↳ Um objeto iluminado c/ luz branca parece-nos de uma certa cor porque ^{absorve} ~~absorve~~ todas as cores exceto a que está a **refletir**

Percepção cor

No olho: No olho, a luz passa pela íris, é focada pelo cristalino e projeta-se na retina

Visão, Escotópica: a níveis baixos luminosidade e não sensível ao comprimento de onda da luz

Fotópica: a níveis mais elevadas luminosidade e sensível ao comprimento de onda da luz

Células sensoras

↳ Bastonetes: permitem visão escotópica ~~que funciona a níveis baixos~~

Cones: permitem visão fotópica → cones vermelhos: λ mais elevadas → 64%

→ cones verdes: λ médios → 33%

→ cones azuis: λ baixas → 2%

Porquê recorrer a métodos de adaptação cor? Melhorar percepção cor pessoas daltônicas

MODELOS COR

Modelo, Definição sistema coordenadas
Definição subespaço cores visíveis

Como guardar a cor?

- Colocar valores RGB no frame buffer
- Guardar RGB numa tabela e guardar no frame buffer o endereço

Modelo cor CIE

- Define 3 cores primárias imaginárias
- As cores visíveis encontram-se dentro dum volume aproximadamente cónico
- Obtenção da cor C: $C = xX + yY + zZ$

Problema: Não é usado na prática, pois não é intuitiva

MODELOS ORIENTADOS PL HARDWARE:

Modelo RGB

- Origem das coordenadas corresponde ao preto
↳ branco obtém-se somando R, G, B
- Modelo aditivo, adequado pl situação em que há emissão luz

Cubo RGB

↳ pode variar de monitor pl monitor
↳ subconjunto cores visíveis pelo olho humano

Modelo CMY

- Origem coordenadas corresponde ao branco
↳ preto obtém-se sobrepondo o branco a C, M, Y d/ valor máximo
- Modelo subtrativo, adequado pl situação em que há reemissão luz
- Cores opacas são reemitidas se existirem no filtro

- Cores primárias RGB são complementares às do CMY
 $C = 1 - R$ $M = 1 - G$ $Y = 1 - B$

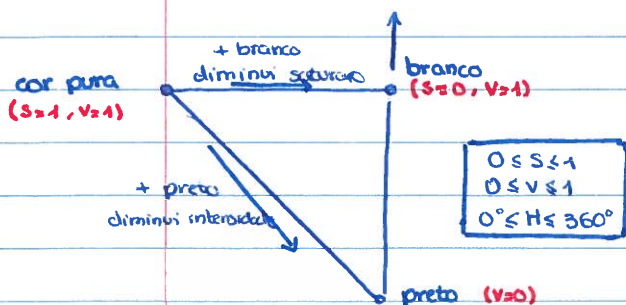
Cor-luz: cores observadas resultam da mistura luzes das 3 cores em %s $\neq$

cor-pigmento: cores observadas resultam da mistura 3 cores em %s $\neq$ e na forma pigmentos

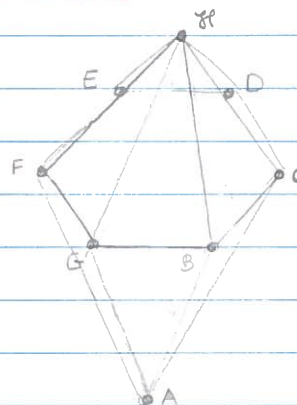
MODELOS ORIENTADOS PI UTI

Modelo HSV - Hue Saturation Value

↳ Reflexo RGB/HSV é não linear
logo interpolação cores nos 2 métodos
não dá o mesmo resultado



Modelo HSL - Hue Saturation Light

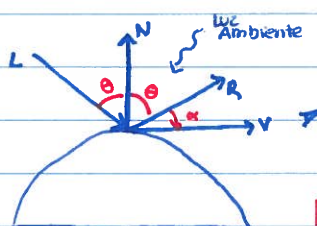


A - Preto	0.0
B - Magenta	
C - Vermelho	0°
D - Yellow	
E - Green	120°
F - Ciano	
G - Azul	240°
H - White	1.0

MODELOS ILUMINAÇÃO LOCAL

Fontes luz pontuais

Componente luz ambiente q
compensar a não consideração das
interações luminosas entre obj's



ILUMINAÇÃO LOCAL Luz ambiente

$$I = I_a K_a$$

Depende do
ângulo θ

$$I = I_{amb} + I_{refl} (I_{diffuse} + I_{specular})$$

Iluminação
ambiente

Depende de
θ e α

Quando luz incide sobre objeto ocorre há reemissão de luz ou por:

Difusão - Não direcional

↳ Superfícies pouco polidas refletem luz c/ intensidade em todas as direções

$$I_d = I_p K_d \cos(\theta)$$

↑ Intensidade fonte luz

↑ intensidade da luz difundida

↑ coeficiente de reflexo difusa

↑ ângulo incidência luz



$$\cos(\theta) = \frac{L \cdot N}{|L| |N|}$$

↓ produto interno

Reflexão

↳ comportamento luz que incide num espelho perfeito
quanto + polida menor a dispersão
brilho depende da pos do observador

$$I_s = I_p K_s [\cos(\alpha)]^n$$

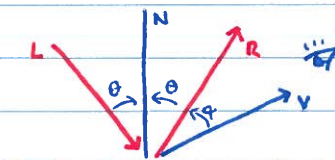
↑ Intensidade fonte luz

↑ intensidade luz refletida

↑ coeficiente de reflexo especular

↑ expoente reflexo especular (1 a ∞)
at > n
+ polido

↑ ângulo entre direção reflexo e visualização
brilho max qd α=0



Se não tivermos em conta a
distância, 2 obj's podem parecer
iguais

Atenuação luz c/ a distância:

$$I_{at} = \frac{1}{d^2}$$

Equação modelo iluminação local:

$$I = I_a K_a + I_p I_{at} [K_d (L \cdot N) + K_s (R \cdot V)^n]$$

Múltiplas fontes de luz

$$I = I_a K_a + \sum_i I_i s_m (I_p f_{oi} [K_d (L_i \cdot N) + K_s (R_i \cdot V)^n])$$

Shading de malhas poligonais

↳ n° poligonos deve ser grande p/ melhor aproximação

Luz reemitida por um elemento depende da orientação da superfície

Shading constante (Flat)

- Atribui-se a mesma cor a todos pixels
- Na transição entre polígonos adjacentes ocorre uma descontinuidade

↓
efeito bandas Mach:

qd há ≠ coloração entre
faces adjacentes os nossos
olhos exageram

Algoritmo:

- Calcular normal exterior p/ cada face da malha poligonal
- Aplicar fórmula modelo iluminação em cada polígono malha poligonal

Shading interpolação intensidades (Gouraud)

Podem perder-se highlights pois apenas se interpolam cores a partir das cores dos vértices

Algoritmo:

- Calcular normal exterior p/ cada face malha poligonal
- Calcular normais em cada vértice interpolando as normais aos polígonos adjacentes ao vértice
- Aplicar fórmula modelo iluminação em cada vértice malha poligonal
- Interpolador valores da intensidade luminosa ao longo das arestas do polígono
- Interpolador valores da intensidade luminosa ao longo da linha de varrimento

Shading w/ interpolação normais (Phong)

Têm highlights porque se calcula a cor iluminada em cada ponto da imagem

- Calcular normal exterior p/ cada face malha poligonal
- Calcular normais em cada vértice interpolando as normais aos polígonos adjacentes ao vértice
- Calcular normais em cada aresta interpolando as normais nos vértices que definem a aresta
- Calcular normais em cada pixel linha varrimento, por interpolação das normais nas arestas que a delimitam
- Aplicar fórmula modelo iluminação em cada pixel da imagem

Modelos Iluminação Global

Ray-casting

- Inverte sentido trajetória raios luz, passando o observador a ser a fonte
- Baseia-se métodos ótica geométrica que determinam percursos raios luz
- Quando se usa P.Perspetiva, raios passam pelo centro pixel

Se existam raios secundários → **Ray-tracing**

↳ Trata explicitamente interações luminosas entre objetos (⇒ modelo iluminação global)

- Eliminação superfícies ocultas
- Shading devido a iluminação direta
- Shading devido a iluminação global
- Determinação sombras

cor do pixel:
soma de todas as contribuições que são calculadas em todos os raios da árvore gerada pelo encaminhamento de raios secundários

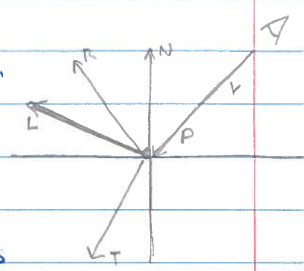
- Algoritmo:**
1. Traçar raio a partir do observador
 2. Interpretar o modelo cena
 3. Determinar ponto interseção + próximo
 4. Calcular iluminação nesse ponto
 5. Atribuir cor ao pixel

Calcular sombras: Lançar raio adicional a partir do ponto interseção p/cada fonte luz. Raio encontra objeto ⇒ P está na sombra

Raio n interseção: Atribui-se cor fundo cena ao pixel por onde raio passa

Raio interseção: É preciso determinar cor do pixel objeto + prox.

- Lançamento raios secundários:**
1. raio n interseção não termina quando
 2. raio interseção fonte de superfície na reflexão
 3. nível profundidade árvore chega ao máximo

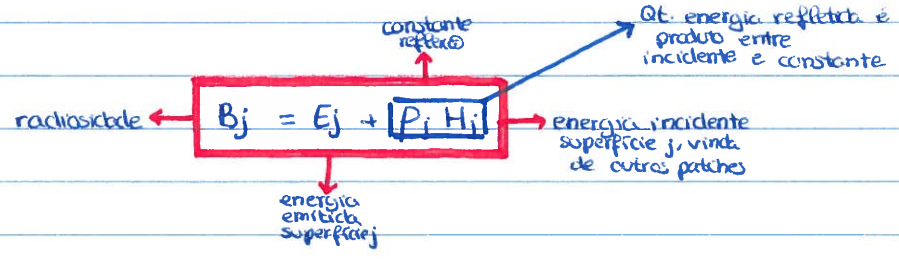


→ **Radiosidade**

↳ Calcula a taxa energia luminosa total que é liberada pelas superfícies

Adequado p/cenas em que predominam superfícies refletoras difusas:

- Energia luminosa incidente numa superfície é refletida $\forall$ = intensidade em todas direções
- Aspeto não depende da pos do observador



RECORTE

↳ P/c determinar as partes duma imagem que estão dentro ou fora de uma determinada região

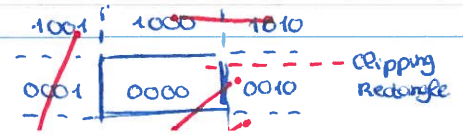
RECORTE SEGMENTOS RETA

→ **Algoritmo Cohen - Sutherland**: Recorte segmentos reta em relap a polígonos recorte retangulares

1ª Fase - Determinar segmentos a recortar

- (0000) **Categoria 1:** Ambos os extremos dentro retângulo trivialmente visível
- (ANDbits ≠ 0000) **Categoria 2:** Ambos os extremos do polígono à esquerda ou à direita trivialmente não visível
- (ANDbits = 0000) **Categoria 3:** Não se encaixam nas outras categorias candidatos a recorte

→ Plano dividido em 9 regiões às quais se atribuem códigos 4 bits



2ª Fase - Recorte dos segmentos candidatos a recorte

- Analisa-se código dum extremo fora área recorte e vê-se qual o 1º bit 1 a partir esquerda
- Calcula-se interseção do lado correspondente ao teste anterior e obtêm-se 2 sub-segmentos

Dos 2 segmentos: Rejeita-se o que é exterior (o que liga P ao ponto interseção)

Calcula-se código ponto interseção

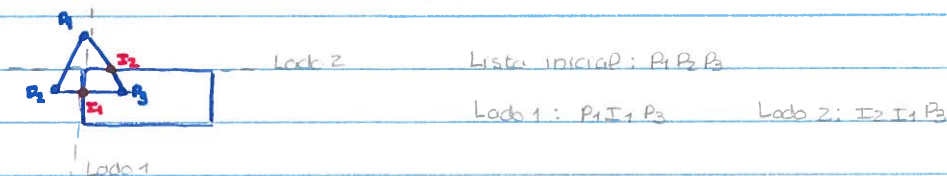
Aplica-se novamente o algoritmo ao sub-segmento não rejeitado

Algoritmo Sutherland - Hodgman

↳ Efetua recorte de polígonos convexos ou côncavos em relação a um polígono recorte convexo

1. Processa o recorte em relação a um lado do polígono de recorte de cada vez, construindo uma nova lista de vértices. Para cada aresta adicionam-se 0, 1 ou 2 vértices à lista
2. Esta nova lista é usada para efetuar o recorte em relação ao lado seguinte

→ Algoritmo termina quando se tiver feito o recorte em relação a todos os lados do polígono de recorte ou quando lista vértices for vazia



Algoritmo de Weiler - Atherton

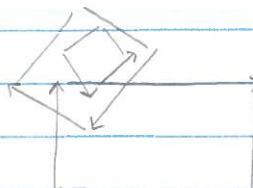
↳ Recorte polígonos côncavos ou convexos, com ou sem buracos

Fronteira exterior: setas em sentido retrógrado (relógio)

Fronteira interior: setas em sentido direto

- Os pontos interseção entre as duas fronteiras são classificados como sendo de entrada ou de saída

- Interseções acontecem aos pares



ERIMINACÃO SUPERFÍCIES INVISÍVEIS

↳ O problema eliminação invisíveis pode ser resolvido num espaço coordenadas tridimensionais c/ a definição objetos

Espaco objetos

- P/ cada objeto determinam-se as partes que estão escondidas pelo próprio objeto ou por outros e gera-se a respetiva imagem

Se quisermos alterar tamanho imagem?

- Cálculos relativos à detecção visibilidade não precisam ser repetidos

Só a transformação imagem p/ ecrã

Complexidade: $O(N^2)$ ^{Nº objetos}

Espaco imagens

- P/ cada pixel determinam-se os objetos cuja projeção cobre o pixel e usa-se o objeto + próximo do observador p/ determinar a cor do pixel

Se quisermos alterar tamanho imagem?

- Necessário repetir todos os cálculos relativos à detecção visibilidade

Complexidade: $O(N \times P)$ ^{Nº objetos} ^{Nº pixels}

Algoritmo Back-Face Culling

Eliminação faces posteriores a objeto: $V_{view} = \text{direção observador}$
 $N = \text{normal exterior face}$
 } Se $V_{view} \cdot N > 0$ então a face é posterior (não visível)

Como obter a normal exterior?

$$Ax + By + Cz + D = 0$$

- Escolher 3 vértices polígono no sentido direto qd se olha lado de fora

$$\vec{N} = V_1V_2 \times V_1V_3 = (V_2 - V_1) \times (V_3 - V_1) = (A, B, C)$$

→ P está sobre plano se $Ax_1 + By_1 + Cz_1 + D = 0$

→ P está à frente plano se $Ax_1 + By_1 + Cz_1 + D > 0$

→ P está atrás plano se $Ax_1 + By_1 + Cz_1 + D < 0$

Algoritmo Z-Buffer

↳ Compara profundidades dos polígonos da cena p/ cada pixel ^{no PP}
 } Necessário guardar, p/ cada vértice, o valor coordenada z

Utilizam-se 2 buffers de tamanho imagem: **Frame buffer**: guarda cor pixel

Z-buffer: guarda profundidade polígono
 ↳ varia entre -1 e 0

1. P/ cada pixel: Profundidade = -1
 Cor = cor fundo

2. Em cada polígono p/ cada pixel: Determinar profundidade z
 ↳ se $z > \text{valor z-buffer}$:
 • atualizar profundidade
 • atualizar cor

Atualizações ao mesmo tempo sempre que se encontra p/ o mesmo pixel um polígono mais próx. do observador

↳ Buffer profundidade tem de ter precisão suficiente p/ que não surjam efeitos estranhos na imagem

↳ Resultados finais dependem da ordem pela qual são processados os polígonos, podendo ser ^{incorretos}

- comportamento constante (p/ poucos ou muitos polígonos)
- bom desempenho mesmo c/ muitos polígonos

→ Algoritmo A-Buffer

↳ Extensão Z-buffer que permite tratar objetos não opacos

Cada pos do buffer profundidade guarda lista ligada polígonos cuja projeção cobre pixel

Cor pixel calculada como média das cores das superfícies que cobrem o pixel

→ Algoritmo Warnock

↳ Explorar coerência áreas

se propriedade verificada num ponto há probabilidade de se verificar na vizinhança

→ áreas suficientemente pequenas imagem estará contida no max, num único polígono visível

- Funciona forma recursiva, reduzindo construção de uma imagem complexa a de múltiplas imagens simples
- Obter áreas nos quais construção de imagem trivial
- Se numa área isso não acontecer:
 - área subdivide-se em 4 subáreas iguais e processo sucede recursivamente

1. Todas polígonos são disjuntos
Área fica a cor fundo

2. Apenas 1 polígono contido
Área preenchida a cor de fundo e depois preenchida com polígono

3. 1 polígono envolvente e nenhum intersectado
Área estubo fica a cor polígono envolvente

4. Envolvente a frente que V não disjuntos
Área estubo preenchida a cor polígono envolvente

→ Algoritmo do Pintor

Espaço objeto { Faz-se ordenação das superfícies de acordo a sua posição segundo o eixo z
Não se determinam exactamente quais as superfícies ou partes das superfícies totalmente visíveis

Espaço imagem { Superfícies são desenhadas começando pelas mais afastadas. As superfícies desenhadas posteriormente, tapam as que foram desenhadas antes

→ Se 2 ou + superfícies se tapam mutuamente podemos ver um ciclo infinito no processo ordenação

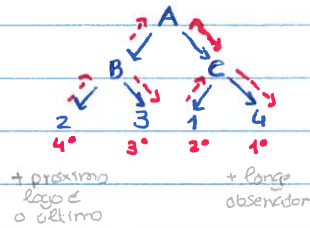
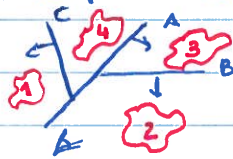
↳ se for detectado um ciclo, deve dividir-se a superfície que provocou o ciclo e ordenam-se separadamente as 2 superfícies resultantes

→ BSP Tree

↳ particpa espaço tridimensional através árvore binária em que cada nó corresponde plano que se divide em 2:

- Em cada nó o plano usado p/ subdividir o espaço:
 - Não tem que ser paralelo a um plano principal
 - Pode ou não corresponder ao plano que contém um dos polígonos da cena. Se este plano intersectar outros polígonos estes serão subdivididos em sub-polígonos, cada um deles pertencente apenas a um dos semi-espaços
- Espaço vai sendo subdividido até que em cada partição haja só 1 objeto
- Plano escolhido p/ nó raiz subdividindo espaço em 2 semi-espaços
 - Subárvore esquerda os objetos e polígonos a frente do plano / normal
 - Subárvore direita os objetos e polígonos atrás do plano / normal

- objetos que estejam no semiespaço + próx observador estão + próximos deste que qualquer objeto contido no outro semiespaço
- desenharam-se os P objetos + afastados do observador e depois os que estão + próximos
- p/ determinar a ordem em que são desenhados os objetos, a BSP-Tree é percorrida em função da pos do observador



- Colocação objetos na BSP é independente pos observador
- Árvore percorrida em função pos observador

Partição Espaço



Quadtree



Cada subárea é dividida em 4 subáreas iguais
linhas // aos eixos principais



Octree



Cada subvolume é dividida em 8 subvolumes iguais
planos // aos planos principais

Partição termina quando cada subárea ou cada sub-volume está vazio ou intersecta um só objeto

→ Bounding Volumes

↳ Volume fechado que envolve um ou + objetos

Podem ter ^{várias} formas:

Paralelepípedo: Bounding Box

AABBs { variable bb
Fixed bb

Esfere: Bounding Sphere

Elipsóide: Bounding Ellipsoid

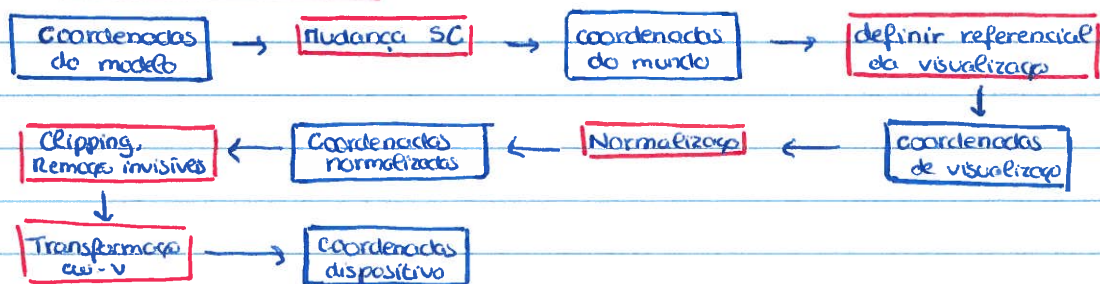
OBBs: A sua orientação é o que melhora se aproxima do objeto que delimita.

Aliasing: efeito "escadado" que surge nas margens das imagens

Anti-Aliasing: minimizar efeito escadado dando a fusão que a retina que delimita a margem é + perfeita

→ Atribui cores intermédias aos pixels vizinhos da fronteira. Assim, a aparência da linha que delimita a fronteira é suavizada.

PIPELINE DE VISUALIZAÇÃO



TEXTURAS :

- 1D:** um padrão p/ colorir uma linha
- 2D:** um padrão p/ colorir uma superfície
- 3D:** um padrão p/ colorir um volume

→ Mip-mapping

↳ Gerar, a partir da mesma textura, texturas mais pequenas

cada uma reduz p/ $1/4$ a textura anterior

Assim, quando textura é aplicada em superfícies a $\pm$ s distâncias, textura exata correta é aplicada.

→ Bump-mapping

↳ Simular rugosidade superfícies introduzindo uma perturbação na normal da superfície

⚠ Análogo ao mip-mapping só que c/ deslocamentos

→ Sprites

↳ imagem que pode mover-se no ecrã

cena concebida através de layers

↳ tem valor profundidade associado
pode ser sprite ou sprite-layer

cena gerada desenhando os sprite-layers de trás p/ a frente

→ Billboards

↳ polígono que se encontra sempre de frente p/ a câmara

Associado sistema eixos (u, r, n): **u:** up vetor fixo p/ objetos "verticais"

n: normal vetor $\perp$ ao polígono

r: produto externo de u e n

- Fundamental p/ obter movimentos de rotação a aplicar o billboard p/ o manter sempre de frente p/ a câmara